

Viva Q&A — PS 26061 · AI/ML focus

AI-Driven Smart Energy Management for Polar Research Stations · MoES / NCPOR · one-line answers

The one framing to hold on to: ML predicts, optimisation decides. If the judge is an AI/ML person, she will probe the model — but the strongest answer you have is *why you chose a small interpretable model on purpose*, not a bigger one.

Say early: "Machine learning tells us what will happen. Optimisation decides what to do. Most teams only build the first half."

1 The model — expect the most questions here

What ML model do you use?	LightGBM — gradient-boosted decision trees, one model per target.
Why LightGBM over a neural network?	Tabular data with strong seasonality; GBDTs beat NNs at this data size and train in seconds.
Why not an LSTM?	LSTMs need far more data than one station-year, and add nothing once lag features exist.
Why not a Transformer?	Same data problem, plus there is no GPU at a polar station.
What exactly do you predict?	Four targets: electrical load, thermal load, solar availability, wind availability.
What is the forecast horizon?	72 hours ahead, at hourly resolution.
Direct or recursive multi-step?	Direct — a separate model per horizon, so errors do not compound.
Regression or classification?	Regression, plus quantile regression for uncertainty.
How do you quantify uncertainty?	Three quantile models — P10, P50, P90 — trained with the pinball loss.
Why do you need P10 and P90?	The optimiser sizes its reserve margin from the uncertainty, not from the mean.
How many models in total?	Four targets × three quantiles, per horizon block.
How big is the model?	A few megabytes; inference is milliseconds on a plain CPU.
How long does training take?	Seconds to a couple of minutes on a laptop — no GPU anywhere in the pipeline.
Why is "small and boring" a feature?	It retrains on-station, runs offline, and every prediction can be explained to the operator.

2 Features

What features go in?	Lags, rolling statistics, weather, occupancy, and calendar encodings.
Which lags?	t-1, t-2, t-24, t-48, t-168 — hourly, daily and weekly memory.
Why a t-168 lag?	Station routines repeat weekly — laundry, workshop days, science schedules.
How do you encode hour and month?	Sine/cosine pairs, so 23:00 and 00:00 sit next to each other instead of far apart.
Any domain-specific features?	Heating degree hours, solar elevation angle, wind chill, crew headcount.
Why solar elevation, not clock time?	It encodes polar night and midnight sun directly; clock time cannot.
How is future occupancy known?	It is scheduled — expedition rosters are fixed months ahead, so it is a known future covariate.
Weather actuals or forecasts?	Forecasts at inference, so we train on forecast-like inputs to avoid optimistic bias.
How many features?	Roughly 40–60 — kept deliberately small against one year of data.
Do you do feature selection?	Yes — SHAP importance plus drop-column checks on a held-out period.

3 Data & training

What do you train on today?	A physics-simulated year from PolarSim, until NCPOR telemetry is available.
Isn't training on synthetic data cheating?	It's a clearly-labelled modelling assumption — we validate the control strategy, not the absolute litres.
Then why trust your numbers?	Baseline and optimiser run in the same simulator, so the <i>percentage</i> gain holds even if the scale is off.
What changes with real data?	Nothing structural — same pipeline, retrained and re-measured.
How much data do you need?	One full annual cycle minimum, because the dominant seasonality is annual.

How do you move to another station?	Fine-tune the Bharati model on the new station's short history instead of training from scratch.
How do you handle missing sensor data?	LightGBM handles NaNs natively; long gaps trigger the rule-based fallback.
What about outliers?	Flagged, not deleted — a blizzard is a real event, not a data error.
Do you scale or normalise?	Not needed — tree models are invariant to monotonic feature scaling.
Class imbalance issues?	Not applicable — it's regression; the analogue is rare extreme events, handled by quantile models.

4 Validation & metrics — where an ML judge will test you hardest

How do you validate?	Rolling-origin (expanding window) time-series cross-validation.
Why not standard k-fold?	Random folds leak the future into the past — on time series that inflates scores and is simply wrong.
What metrics do you report?	MAE and RMSE for the point forecast, pinball loss for the quantiles.
Why not MAPE?	Loads near zero make MAPE explode; we report it but never rely on it.
What is your baseline?	Seasonal-naive persistence — same hour yesterday, same hour last week.
Why compare to a baseline at all?	An error figure is meaningless until you know what doing nothing would have cost.
How do you prevent overfitting?	Early stopping on a later held-out period, shallow trees, few features.
Hyperparameter tuning?	Small random search under time-series CV — learning rate, num leaves, min child samples.
How do you check calibration?	Coverage test — roughly 80 % of actuals should fall inside the P10–P90 band.
What if coverage is wrong?	The bands are miscalibrated, so reserve is mis-sized — we recalibrate before trusting the optimiser.
Any data leakage risks?	Yes — using weather actuals as if they were forecasts; we avoid it deliberately.

5 Why not deep learning or RL — she will almost certainly ask

Why not reinforcement learning for control?	RL cannot guarantee hard safety constraints, and this is a life-safety system.
But RL might beat your MILP?	On average, maybe — average isn't the bar; we need the guarantee to hold every hour.
Would you ever use RL?	As a warm-start heuristic to speed the solver, never as the final authority.
Why not end-to-end deep learning?	No dataset, no GPU, no explainability, no constraint guarantees — four independent blockers.
Is a MILP even "AI"?	Yes — constrained optimisation and search are classical AI; ML predicts, optimisation decides.
So where is the AI in your project?	In both halves: learned forecasting models, and a constrained optimiser making the decisions.
Isn't that just a rules engine?	No — the rules engine is only the safety fallback; the optimiser searches a large combinatorial space.
Would a bigger model help?	Not with one year of data — we'd be fitting noise, and we'd lose explainability we actually need.

6 Explainability & production ML

How do you explain a prediction?	SHAP values, showing which features drove that specific forecast.
How do you explain a decision?	The optimiser reports the binding constraint and the litres saved versus baseline.
Why does explainability matter here?	The engineer must be able to override — and nobody overrides what they cannot understand.
Which features dominate?	Temperature and crew size for thermal; hour-of-day and occupancy for electrical — confirmed by SHAP, not assumed.
Do you retrain in production?	Yes — rolling retrain, triggered when residual drift crosses a threshold.

What is concept drift here?	New equipment, a changed crew size, a rebuilt module — the load shape shifts underneath the model.
How do you monitor the model?	Track rolling MAE and P10–P90 coverage; alert the operator when either degrades.
What if the model degrades badly?	Fall back to the seasonal-naive forecast and the rule-based dispatcher — the station stays safe.
Cold start at a new station?	Run on physics + persistence until enough history accumulates, then switch to the learned model.

7 How ML connects to the solution

How does ML feed the optimiser?	Forecasts become the right-hand side of the optimiser's constraints for the next 48 hours.
What if the forecast is wrong?	We re-optimize every hour, so drift is corrected before it compounds.
Do forecast errors destroy the savings?	They cost some — we measure and report that, instead of quoting perfect-foresight numbers.
Which matters more, forecast or optimiser?	The optimiser — a perfect forecast with naive dispatch still wastes fuel.
Can you prove that?	Yes — we run the optimiser under perfect foresight as an upper bound and compare against it.
What does the optimiser actually decide?	Which generator runs and at what loading, battery charge/discharge, and when flexible loads start.
Why MILP and not LP?	Generator on/off and load start times are binary decisions — that makes it mixed-integer.
How fast does it solve?	Seconds on a CPU — 48 hourly steps and a handful of assets.

8 The solution in one line each

What does the system do?	Predicts the next 72 hours, plans the next 48, and reports how many days of fuel remain.
What is the core insight?	Generator waste heat warms the station — so electricity and heat must be optimised together.
Why can't you just switch the generator off?	You lose the waste heat and must fire the boiler — often burning more fuel than you saved.
What is your hero metric?	Days of autonomy remaining before the next resupply ship.
Why not minimise cost?	There is no electricity price in Antarctica — survival margin is the real currency.
What is new versus existing EMS?	They optimise electricity reactively; we co-optimize heat and electricity predictively.
What happens if everything fails?	A deterministic rule-based dispatcher takes over and always returns a safe, feasible schedule.
Does it run offline?	Fully — CPU only, no cloud, no GPU, no satellite dependency.
What's the weakest part?	The data — every number is a labelled projection until NCPOR shares real telemetry.

9 Lines worth memorising

"Machine learning tells us what will happen. Optimisation decides what to do. Most teams only build the first half."

"We chose the small model on purpose — it trains in seconds, runs on a CPU at the station, and we can explain every prediction to the engineer who has to trust it."

"There is no electricity price in Antarctica. We don't minimise cost — we minimise the chance of running out."

"We deliberately did not use deep RL. You cannot promise a neural policy will never drop life support, and in July at Bharati that promise is the whole product."

"If she asks for our biggest weakness: the data. We say projected on every number, and we mean it."